

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: HỌC MÁY

TÊN ĐỀ TÀI: XÂY DỰNG HỆ THỐNG NHẬN
DIỆN BIẾN BÁO GIAO THÔNG

Khóa:	K17
Ngành:	KHMT(AI&DL)
Hệ:	Chính quy
Giảng viên hướng dẫn:	PGS.TS Phạm Tiến Lâm

Sinh viên thực hiện:

Mầu Danh Chiến – 23017270

Trần Minh Cường – 23017311

Nguyễn Thành Đạt – 23017259

Đinh Long Vũ – 23017241

Hà Nội – 2026

Lời mở đầu

Học máy (Machine Learning) là một lĩnh vực quan trọng của Trí tuệ nhân tạo (Artificial Intelligence – AI), tập trung nghiên cứu các thuật toán và mô hình có khả năng học từ dữ liệu để đưa ra dự đoán hoặc quyết định mà không cần được lập trình chi tiết cho từng trường hợp. Với sự phát triển mạnh mẽ của Big Data, năng lực tính toán và Học sâu, Học máy đã đạt được nhiều thành tựu nổi bật, góp phần thúc đẩy sự phát triển của các hệ thống thông minh.

Ngày nay, Học máy được ứng dụng rộng rãi trong nhiều lĩnh vực như nhận dạng hình ảnh, nhận dạng giọng nói, xử lý ngôn ngữ tự nhiên, y tế, tài chính, thương mại điện tử, an ninh mạng, xe tự hành và robot. Những ứng dụng này giúp nâng cao hiệu quả xử lý dữ liệu, hỗ trợ con người đưa ra quyết định chính xác hơn và tự động hóa nhiều công việc trong đời sống cũng như sản xuất.

Việc nghiên cứu Học máy không chỉ giúp hiểu rõ nguyên lý hoạt động của các hệ thống trí tuệ nhân tạo mà còn trang bị nền tảng kiến thức để xây dựng các mô hình dự đoán và giải quyết những bài toán thực tiễn. Đây là một lĩnh vực có tiềm năng phát triển mạnh mẽ, đóng vai trò quan trọng trong cuộc Cách mạng công nghiệp 4.0 và xu hướng chuyển đổi số trên toàn thế giới.

Mục lục

CHƯƠNG 1: GIỚI THIỆU	6
1.1 Bối cảnh và động lực nghiên cứu	6
1.2 Mục tiêu bài tập lớn	7
1.3 Cấu trúc báo cáo	7
CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT	8
2.1 Mạng nơ-ron nhân tạo (ANN)	8
2.2 Mạng nơ-ron tích chập (CNN)	8
2.2.1 Phép tích chập (Convolution)	9
2.2.2 Lớp Pooling	9
2.2.3 Batch Normalization	10
2.2.4 Dropout	10
2.2.5 Hàm Softmax và phân loại đa lớp	10
2.2.6 Hàm mất mát Categorical Cross-Entropy	11
2.3 Thuật toán tối ưu Adam	11
2.4 Data Augmentation (Tăng cường dữ liệu)	11
CHƯƠNG 3: XỬ LÝ ẢNH VỚI OPENCV	13
3.1 Tổng quan về OpenCV	13
3.2 Không gian màu BGR và RGB	13
3.3 Quản lý vùng quan tâm (Region of Interest)	14
3.4 Hiển thị kết quả màn hình	15
CHƯƠNG 4: MÔ HÌNH HUẤN LUYỆN. PHÂN TÍCH MODEL	16
4.1 Tổng quan về model.py	16
4.2 Cài đặt thư viện và khởi tạo	16
4.3 Tiền xử lý dữ liệu và Data Augmentation	17
4.4 Kiến trúc mô hình CNN	20
4.5 Biên dịch mô hình và chiến lược huấn luyện	23
4.6 Luồng dữ liệu qua mô hình	25
CHƯƠNG 5: ỨNG DỤNG NHẬN DIỆN THỜI GIAN THỰC. PHÂN TÍCH MAIN	26
5.1 Tổng quan về main.py	26
5.2 Định nghĩa tập nhãn (class labels)	26
5.3 Tải mô hình đã huấn luyện	27
5.4 Khởi tạo và cấu hình webcam	28

5.5	Vòng lặp chỉnh xử lý thời gian thực.....	28
5.6	Sơ đồ luồng xử lý toàn hệ thống.....	32
CHƯƠNG 6: ĐÁNH GIÁ KẾT QUẢ VÀ THỬ NGHIỆM.....		33
6.1	Các chỉ số đánh giá hiệu suất mô hình.....	33
6.2	Kết quả huấn luyện dự kiến.....	34
6.3	Hiệu suất mô hình theo từng loại biến báo.....	35
6.4	Ảnh hưởng của ngưỡng tin cậy	36
6.5	Phân tích hiệu năng thời gian thực	36
6.6	Phân tích lỗi phổ biến.....	37
CHƯƠNG 7: MÔ HÌNH HỌC MÁY SVM VÀ ViT		38
7.1	Mô hình học máy Support Vector Machine (SVM)	38
7.1.1	Giới thiệu về Support Vector Machine.	38
7.1.2	Áp dụng SVM vào bài toán nhận diện Biên Báo Giao Thông.	38
7.2	Mô hình Vision Transformer (ViT)	40
7.2.1	Giới thiệu mô hình Vision Transformer (ViT).....	40
7.2.2	Cấu trúc và Kiến trúc chi tiết của ViT	40
7.2.3	Phương pháp Fine-Tuning và Chuyển giao Tri thức (Transfer Learning)	41
CHƯƠNG 8: SO SÁNH VÀ ĐÁNH GIÁ CÁC MÔ HÌNH		42
8.1	Môi trường thực nghiệm và tập dữ liệu.....	42
8.1.1	Phần mềm và thư viện.....	42
8.1.2	Dữ liệu thực nghiệm	42
8.2	Triển khai các mô hình.....	42
8.2.1	Mô hình Support Vector Machine SVM.....	42
8.2.2	Mô hình Mạng thần kinh tích chập CNN.....	42
8.2.3	Mô hình Vision Transformer ViT.....	42
8.3	Kết quả và bảng so sánh.....	43
8.4	Đánh giá và so sánh	43
Chương 9 : Kết luận và hướng phát triển		44
9.1	Mô hình tốt nhất	44
9.2	Kết quả đạt được	44
9.3	Đánh giá ưu điểm và hạn chế	45
9.4	Hướng phát triển.....	46
9.5	Kết luận.....	46
Tài liệu tham khảo		48

DANH MỤC HÌNH ẢNH

Hình 1.1: Sơ đồ tổng quát nhận diện hệ thống biển báo giao thông	6
Hình 2.1: Phép tích chập trên ảnh 2 chiều với bộ lọc 3×3	9
Hình 2.2: Phép Max Pooling kích thước 2×2 với stride=2	9
Hình 4.1: Luồng dữ liệu qua các lớp của mô hình CNN	25
Hình 5.1: Sơ đồ luồng xử lý của ứng dụng thời gian thực	32
Hình 6.1: Đồ thị độ chính xác qua các epoch huấn luyện	34
Hình 6.2: Đồ thị hàm mất mát qua các epoch huấn luyện	34

DANH MỤC BẢNG

Bảng 2.1: Các phép biến đổi data augmentation được sử dụng.....	12
Bảng 4.1: Chi tiết các tham số ImageDataGenerator	18
Bảng 4.2: Tóm tắt các lớp và số tham số.....	23
Bảng 5.1: Chuỗi tiền xử lý ảnh	30
Bảng 6.1: Hiệu suất mô hình theo từng loại biến báo	35
Bảng 6.2: Hiệu suất hệ thống theo ngưỡng tin cậy khác nhau	36
Bảng 6.3: Hiệu năng phân cứng trên hệ thống thử nghiệm	36
Bảng 6.4: Các trường hợp lỗi thường gặp và nguyên nhân	37
Bảng 8.1: Bảng so sánh hiệu năng giữa các mô hình Học máy.....	43

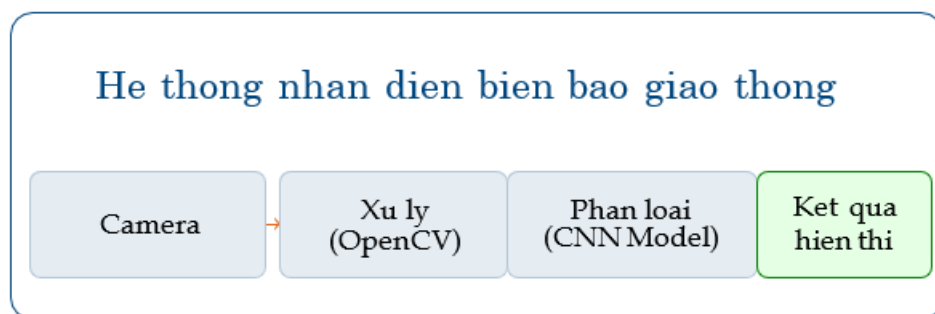
CHƯƠNG 1: GIỚI THIỆU

1.1 Bối cảnh và động lực nghiên cứu

Báo cáo này được thực hiện trong khuôn khổ học phần Học Máy, nhằm nghiên cứu và xây dựng một hệ thống nhận diện biển báo giao thông tự động sử dụng mạng nơ-ron tích chập (Convolutional Neural Network – CNN) kết hợp với thư viện xử lý ảnh OpenCV. Hệ thống được huấn luyện trên tập dữ liệu ảnh biển báo giao thông thực tế, có khả năng phân loại 38 loại biển báo khác nhau theo tiêu chuẩn giao thông Việt Nam.

Giao thông đường bộ ngày càng trở nên phức tạp và đòi hỏi người điều khiển phương tiện phải xử lý nhiều thông tin trực quan trong thời gian ngắn. Biển báo giao thông là kênh truyền thông tin pháp lý và an toàn quan trọng nhất trên đường, nhưng khả năng nhận biết biển báo có thể bị giảm sút do mỏi mệt, mất tập trung, điều kiện thời tiết xấu hoặc ban đêm. Theo thống kê của Ủy ban An toàn Giao thông Quốc gia Việt Nam, một tỷ lệ đáng kể các vụ tai nạn giao thông liên quan đến việc không tuân thủ biển báo hiệu giao thông.

Xuất phát từ thực tiễn đó, việc xây dựng một hệ thống hỗ trợ lái xe có khả năng tự động nhận biết biển báo trong thời gian thực là bài toán có ý nghĩa thiết thực cao, phù hợp với xu hướng phát triển xe tự lái (autonomous vehicles) và hệ thống hỗ trợ lái xe tiên tiến (Advanced Driver Assistance Systems-ADAS).



Hình 1.1: Sơ đồ tổng quát nhận diện hệ thống biển báo giao thông

1.2 Mục tiêu bài tập lớn

Mục tiêu chính của bài tập lớn này bao gồm:

1. Nghiên cứu lý thuyết: Tìm hiểu cơ chế hoạt động của mạng nơ-ron tích chập (CNN), các kỹ thuật tiền xử lý ảnh bằng OpenCV và các phương pháp tăng cường dữ liệu (data augmentation).
2. Xây dựng mô hình: Thiết kế và huấn luyện mô hình CNN với kiến trúc phù hợp cho bài toán phân loại biển báo giao thông 38 lớp.
3. Triển khai ứng dụng: Xây dựng ứng dụng nhận diện thời gian thực sử dụng webcam, sử dụng OpenCV để thu thập và tiền xử lý ảnh đầu vào trước khi đưa vào mô hình.
4. Đánh giá kết quả: Phân tích hiệu suất mô hình qua các chỉ số đo lường và điều chỉnh siêu tham số để tối ưu hóa.

Phạm vi thực hiện

1. Ngôn ngữ lập trình: Python 3.x
2. Thư viện chính: TensorFlow/Keras, OpenCV, NumPy
3. Tập dữ liệu: 38 lớp biển báo, được chia 80% huấn luyện / 20% xác thực
4. Kích thước ảnh đầu vào: $30 \times 30 \times 3$ pixel (RGB)
5. Phương tiện thử nghiệm: Webcam 640×480 pixels

1.3 Cấu trúc báo cáo

Báo cáo được tổ chức theo cấu trúc sau:

Chương 1 : Giới thiệu.

Chương 2 : Trình bày cơ sở lý thuyết về mạng nơ-ron tích chập.

Chương 3 : Giải thích các kỹ thuật xử lý ảnh với OpenCV.

Chương 4 : Mô tả kiến trúc mô hình và quá trình huấn luyện (model.py).

Chương 5 : Trình bày ứng dụng nhận diện thời gian thực (main.py).

Chương 6 : Đánh giá kết quả và thử nghiệm.

Chương 7 : Mô hình SVM và ViT.

Chương 8 : So sánh mô hình CNN với SVM và ViT.

Chương 9 : Tổng kết kết quả và đề xuất hướng phát triển.

CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT

2.1 Mạng nơ-ron nhân tạo (ANN)

Mạng nơ-ron nhân tạo (Artificial Neural Network – ANN) là mô hình tính toán được cảm hứng từ cơ chế hoạt động của não người. Mạng gồm các lớp nơ-ron liên kết với nhau, trong đó giá trị đầu ra của mỗi nơ-ron được tính qua hàm kích hoạt áp lên tổng trọng số của các đầu vào.

Cho một nơ-ron trong lớp ẩn, đầu ra a được tính như sau:

$$a = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

Trong đó x_i là đầu vào, w_i là trọng số, b là sai số (bias), và $f(\cdot)$ là hàm kích hoạt. Trong bài tập này, hàm kích hoạt chính được sử dụng là ReLU:

$$f(z) = \max(0, z) \quad (2.2)$$

2.2 Mạng nơ-ron tích chập (CNN)

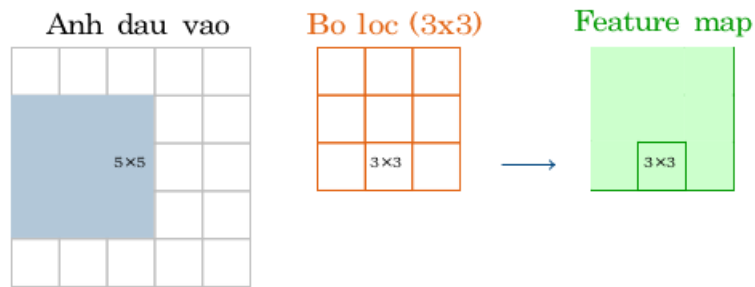
Mạng nơ-ron tích chập (Convolutional Neural Network – CNN) là một kiến trúc đặc biệt của mạng nơ-ron sâu, được thiết kế tối ưu cho việc xử lý dữ liệu có cấu trúc lưới (grid-like data), đặc biệt là ảnh số. Ý tưởng này được thực hiện bởi LeCun et al. (1998) trong nghiên cứu nền tảng về phân loại ảnh viết tay.

2.2.1 Phép tích chập (Convolution)

Lớp tích chập là thành phần chính của CNN. Trong không gian hai chiều, phép tích chập được định nghĩa như sau:

$$(I * K)[i, j] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} I[i + m, j + n] \cdot K[m, n]$$

trong đó I là ma trận ảnh đầu vào, K là bộ lọc (kernel/ lter) kích thước $M \times N$, và $[i, j]$ là tọa độ pixel đầu ra.



Hình 2.1: Phép tích chập trên ảnh 2 chiều với bộ lọc 3×3

Các bộ lọc trong CNN được học tự động trong quá trình huấn luyện, thay vì được định nghĩa thủ công như trong xử lý ảnh truyền thống. Mỗi bộ lọc phụ trách phát hiện một loại đặc trưng nhất định.

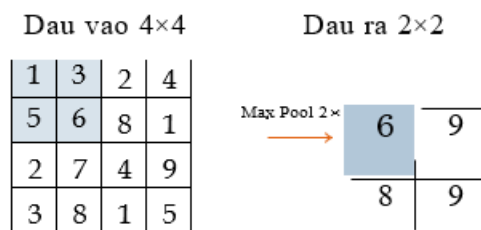
2.2.2 Lớp Pooling

Sau lớp tích chập, lớp Pooling được sử dụng để giảm kích thước không gian của feature map, giúp giảm số lượng tham số và phép tính trong các lớp tiếp theo, đồng thời tăng khả năng bất biến đối với phép tịnh tiến (translation invariance).

Phép Max Pooling với cửa sổ kích thước $f \times f$ và bước trượt (stride) s được định nghĩa:

$$O[i, j] = \max_{(m, n) \in W_{ij}} I[m, n] \quad (2.4)$$

trong đó W_{ij} là cửa sổ pooling tại vị trí (i, j) .



Hình 2.2: Phép Max Pooling kích thước 2×2 với $\text{stride}=2$

2.2.3 Batch Normalization

Batch Normalization (BN) là kỹ thuật chuẩn hóa đầu vào của mỗi lớp trong mạng nơ-ron theo từng mini-batch. Cho một mini-batch $B = x_1, \dots, x_m$, Batch Normalization thực hiện:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i \quad (2.5)$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad (2.6)$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad (2.7)$$

$$y_i = \gamma \hat{x}_i + \beta \quad (2.8)$$

Trong đó γ và β là các tham số học được (scale và shift), ϵ là hằng số nhỏ nhằm tránh phép chia cho 0. Batch Normalization giúp ổn định quá trình huấn luyện, giảm hiện tượng Internal Covariate Shift và cho phép sử dụng tốc độ học lớn hơn.

2.2.4 Dropout

Dropout là kỹ thuật chính quy hóa (regularization) hiệu quả cho mạng nơ-ron. Trong quá trình huấn luyện, mỗi nơ-ron được tắt ngẫu nhiên với xác suất p :

$$\tilde{h}_i = \text{mask}_i \cdot h_i, \quad \text{mask}_i \sim \text{Bernoulli}(1 - p) \quad (2.9)$$

Trong giai đoạn suy luận (inference), đầu ra thường được nhân với $(1-p)$ để bù trừ. Dropout giúp giảm hiện tượng overfitting bằng cách buộc mạng học các biểu diễn dư thừa và độc lập hơn.

2.2.5 Hàm Softmax và phân loại đa lớp

Trong bài toán phân loại K lớp, lớp đầu ra sử dụng hàm kích hoạt Softmax:

$$P(y = k | x) = \text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad (2.10)$$

Trong đó z_k là logit (đầu ra trước kích hoạt) của lớp k . Hàm Softmax đảm bảo tổng các xác suất bằng 1 và mỗi xác suất nằm trong khoảng $[0,1]$.

2.2.6 Hàm mất mát Categorical Cross-Entropy

Hàm mất mát được sử dụng cho bài toán phân loại đa lớp là Categorical Cross-Entropy:

$$L = -\frac{1}{N} \sum_{n=1}^N \sum_{k=1}^K y_{nk} \log(\widehat{p}_{nk})$$

Trong đó $y_{nk} \in 0,1$ là nhãn thực (one-hot encoding) của mẫu n tại lớp k , và $\widehat{p}_{nk}$ là xác suất dự đoán tương ứng. Hàm này đo lường sự khác biệt giữa phân phối dự đoán và phân phối thực tế.

2.3 Thuật toán tối ưu Adam

Adam (Adaptive Moment Estimation) là thuật toán tối ưu kết hợp giữa Momentum và RMSProp. Tại bước thời gian t , quy tắc cập nhật tham số là:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (2.12)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (2.13)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (2.14)$$

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t \quad (2.15)$$

trong đó g_t là gradient tại bước t , η là tốc độ học, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$ là các siêu tham số mặc định.

2.4 Data Augmentation (Tăng cường dữ liệu)

Data Augmentation là kỹ thuật tăng số lượng mẫu huấn luyện bằng cách áp dụng các biến đổi hình học và quang học lên ảnh gốc, tạo ra các biến thể mới trong khi vẫn giữ nguyên nhãn. Các phép biến đổi thường dùng bao gồm:

Bảng 2.1: Các phép biến đổi data augmentation được sử dụng

Phép biến đổi	Tham số	Mục đích
Xoay ảnh (Rotation)	$\pm 20^\circ$	Invariant với biên biến báo nghiêng
Tịnh tiến ngang (Width shift)	15%	Biên biến báo không chỉnh tâm
Tịnh tiến dọc (Height shift)	15%	Biên biến báo không chỉnh tâm
Phóng to/thu nhỏ (Zoom)	$\pm 20\%$	Biên biến báo ở các khoảng cách khác nhau
Tang/giam độ sáng (Brightness)	[0.4, 1.6]	Điều kiện ánh sáng khác nhau
Fill mode	Nearest	Bù điểm ảnh bị thiếu
Chuẩn hóa (Rescale)	1/255	Đưa về khoảng [0, 1]

CHƯƠNG 3: XỬ LÝ ẢNH VỚI OPENCV

3.1 Tổng quan về OpenCV

OpenCV (Open Source Computer Vision Library) là thư viện xử lý ảnh và thị giác máy tính mã nguồn mở lớn nhất thế giới, được phát triển ban đầu bởi Intel và hiện nay được duy trì bởi cộng đồng. Thư viện cung cấp hơn 2500 thuật toán được tối ưu cho xử lý ảnh, phát hiện đối tượng, nhận diện khuôn mặt và nhiều ứng dụng xử lý ảnh khác.

Trong dự án này, OpenCV được sử dụng cho hai mục đích chính:

1. Thu thập ảnh từ webcam theo thời gian thực (cv2.VideoCapture).
2. Tiền xử lý ảnh trước khi đưa vào mô hình CNN (chuyển đổi màu, resize, chuẩn hóa).

3.2 Không gian màu BGR và RGB

OpenCV lưu trữ ảnh trong không gian màu BGR (Blue – Green – Red) thay vì RGB (Red – Green – Blue) là chuẩn phổ biến hơn. Sự khác biệt này có thể gây ra lỗi sai màu nếu không được xử lý đúng. Trong dự án này, bước chuyển đổi màu được thực hiện trước khi đưa ảnh vào mô hình :

```
img_rgb = cv2.cvtColor(img_roi, cv2.COLOR_BGR2RGB)
```

Listing 3.1: Chuyển đổi không gian màu từ BGR sang RGB

Lý do phải thực hiện bước này là vì mô hình CNN được huấn luyện với ảnh RGB (thông qua ImageDataGenerator của Keras đọc từ thư mục theo thứ tự R–G–B), nên khi dự đoán, ảnh đầu vào cũng phải ở định dạng RGB để đảm bảo tính nhất quán.

3.3 Quản lý vùng quan tâm (Region of Interest)

Để tập trung nhận diện biển báo tại một vị trí xác định trong khung hình, hệ thống sử dụng kỹ thuật Region of Interest (ROI). ROI được xác định là một hình chữ nhật nhất định trong ảnh toàn phần:

```
1 # Toa do khung ROI (pixel)
2 x_start, y_start = 220, 140
3 x_end, y_end     = 420, 340
4
5 # Ve khung hình chu nhat chi dan nguoi dung
6 cv2.rectangle(img_original,
7               (x_start, y_start),
8               (x_end, y_end),
9               (255, 0, 0), 2) # Mau xanh duong, day net 2px
10
11 # Trich xuất vùng ROI tu anh goc
12 img_roi = img_original[y_start:y_end, x_start:x_end]
```

Listing 3.2: Định nghĩa và trích xuất nguồn ROI

Vùng ROI có kích thước 200×200 pixel (từ (220,140) đến (420, 340)), được đặt ở giữa ảnh webcam có độ phân giải 640×480 .

3.4 Hiển thị kết quả màn hình

```
1 # Hien thi chu dan
2 cv2.putText(img_original,
3             "Dat bien bao vao day", (
4             x_start, y_start - 10), cv2.
5             FONT_HERSHEY_SIMPLEX,
6             0.6,                # Scale font
7             (255, 0, 0),        # Mau xanh duong
8             2)                  # Do day net chu
9
10 # Hien thi ket qua du
11 doan cv2.putText (
12     img_original ,
13     txt ,
14     (20, 50) ,
15     cv2.FONT_HERSHEY_SIMPLEX ,
16     color
17     , 2)
18
19 cv2.imshow("Nhan dien bien bao", img_original)
```

Listing 3.3: Vẽ khung ROI và hiển thị nhãn

CHƯƠNG 4: MÔ HÌNH HUẤN LUYỆN. PHÂN TÍCH MODEL

4.1 Tổng quan về model.py

File model.py là nhân cốt của dự án, chịu trách nhiệm:

1. Tải và tiền xử lý tập dữ liệu biên báo giao thông.
2. Xây dựng kiến trúc mạng CNN.
3. Huấn luyện mô hình và lưu kết quả tốt nhất.

File này sử dụng các thư viện TensorFlow/Keras, NumPy và ImageDataGenerator để thực hiện toàn bộ quá trình từ đọc dữ liệu đến huấn luyện mô hình.

4.2 Cài đặt thư viện và khởi tạo

```
1 import numpy as np
2 import tensorflow as tf
3 from tensorflow.keras.preprocessing.image import
   ImageDataGenerator
4 from tensorflow.keras.models import Sequential
5 from tensorflow.keras.layers import (Conv2D, MaxPooling2D,
6                                     BatchNormalization,
7                                     Flatten, Dense, Dropout)
8 from tensorflow.keras.optimizers import Adam
9 import os
```

Listing 4.1: Import thư viện cần thiết

Giải thích từng dòng:

NumPy (numpy as np): Thư viện tính toán ma trận, dùng để xử lý ảnh và thực hiện các phép tính số học.

TensorFlow (tensorflow as tf): Framework học sâu chính được sử dụng để xây dựng và huấn luyện mô hình.

ImageDataGenerator: Công cụ dùng để đọc ảnh theo từng lô (batch) và áp dụng các kỹ thuật *data augmentation*.

Sequential: Mô hình CNN trong đó các lớp được xếp chồng theo thứ tự tuyến tính.

Conv2D, MaxPooling2D, ...: Các lớp cơ bản cấu thành mạng CNN.

Adam: Thuật toán tối ưu được sử dụng trong quá trình huấn luyện mô hình.

os: Thư viện dùng để tương tác với hệ thống tập tin (ví dụ: kiểm tra sự tồn tại của thư mục).

Kiểm tra và tải dữ liệu

```
1 # Đường dẫn đến thư mục chứa dữ liệu huấn luyện
2 dataset_path = r"D:\Traffic_Sign_Project\Data"
3
4 # Kiểm tra sự tồn tại của thư mục
5 if not os.path.exists(dataset_path):
6     print("Không tìm thấy dataset:", dataset_path)
7     exit()
```

Listing 4.2: Kiểm tra đường dẫn tập dữ liệu

Khai báo biến `dataset_path` sử dụng *raw string* (tiền tố `r`) để tránh việc xử lý nhầm dấu gạch chéo ngược (`\`) trên hệ điều hành Windows. Hàm `os.path.exists()` được sử dụng để kiểm tra sự tồn tại của thư mục trước khi tiến hành các bước tiếp theo, giúp tránh lỗi runtime khi đường dẫn không đúng.

4.3 Tiền xử lý dữ liệu và Data Augmentation

ImageDataGenerator cho tập huấn luyện

```

1 datagen = ImageDataGenerator (
2     rescale=1./255,          # (1) Chuan hoa pixel ve [0,1]
3     validation_split=0.2,    # (2) Tach 20% lam tap xac thuc
4     rotation_range=20,      # (3) Xoay ngau nhien toi da 20
        do
5     width_shift_range=0.15,  # (4) Tinh tien ngang toi da 15%
6     height_shift_range=0.15, # (5) Tinh tien doc toi da 15%
7     zoom_range=0.2,         # (6) Phong to/thu nho toi da 20%
    brightness_range=[0.4,1.6], # (7) Bien do sang tu 40% den
    fill_mode='nearest'      # (8) Dien pixel bi thieu bang
        gia tri gan nhat
    )

```

Listing 4.3: Khởi tạo ImageDataGenerator với data augmentation

Giải thích chi tiết từng tham số:

Bảng 4.1: Chi tiết các tham số ImageDataGenerator

Tham số	Giá trị	Lý giải
rescale	1/255	Chuyển pixel từ [0,255] về [0,1], giúp ổn định gradient
validation_split	0.2	Dành 20% dữ liệu làm tập xác thực tự động
rotation_range	20	Biến báo trong thực tế có thể bị nghiêng mưa, gió
width_shift_range	0.15	Biến báo không phải lúc nào cũng nằm giữa khung ảnh
height_shift_range	0.15	Tương tự theo chiều dọc
zoom_range	0.2	Biến báo gần/xa ở khoảng cách khác nhau
brightness_range	[0.4, 1.6]	Biên biến báo điều kiện ánh sáng khác nhau (sáng,tối)
fill_mode	nearest	Bù pixel bằng giá trị pixel gần nhất, tự nhiên nhất cho biến báo

Tạo generator cho tập huấn luyện và xác thực

```
1 # Generator cho tap huan luyen (80% du lieu)
2 train_generator = datagen.flow_from_directory(
3     dataset_path,          # Thu muc goc chua tung lop bien
4     target_size=(30, 30),  # Resize moi anh ve 30x30 pixel
5     batch_size=32,         # Moi buoc huan luyen dung 32 anh
6     class_mode='categorical', # Phan loai da lop (one-hot
7     encoding)              # Chi lay phan huan luyen
8 )
9
10 # ImageDataGenerator cho tap xac thuc (chi chuan hoa, khong
11    augment)
12 valid_datagen = ImageDataGenerator(
13     rescale=1./255,
14     validation_split=0.2
15 )
16
17 # Generator cho tap xac thuc (20% du lieu)
18 valid_generator = valid_datagen.flow_from_directory(
19     dataset_path,
20     target_size=(30, 30),
21     batch_size=32,
22     class_mode='categorical',
23     subset='validation'
```

Listing 4.4: Tạo train generator và validation generator

Điểm quan trọng: Tập xác thực sử dụng `valid_datagen` riêng biệt, chỉ thực hiện phép `rescale` mà không áp dụng *data augmentation*. Điều này là chính xác về mặt thiết kế vì tập xác thực phải phản ánh dữ liệu thực tế và không bị biến đổi.

Cấu trúc thư mục dữ liệu yêu cầu:

```
1 D:\Traffic_Sign_Project\Data
2 |-- 0\          # Lop 0: Speed limit (20km/
3 |   |--      h)
4 |   |-- 00000. png
5 |   |-- 00001. png
6 |   ...
7 |-- 1\          # Lop 1: Speed limit (30km/h)
8 |   ...
9 |-- 37\         # Lop 37: Di ben trai
10 |   ...
```

Listing 4.5: Cấu trúc thư mục dữ liệu

4.4 Kiến trúc mô hình CNN

Tổng quan kiến trúc

```
1 model = Sequential ([
2     # === BLOCK 1: Trích xuất đặc trưng bậc thấp ===
3     Conv2D (32, (3,3), activation='relu',
4             input_shape=(30,30,3)), # Lop dau vao
5     BatchNormalization (),
6     Conv2D (32, (3,3), activation='relu'),
7     MaxPooling2D (2,2),
8     Dropout (0.25),
9
10    # === BLOCK 2: Trích xuất đặc trưng bậc cao ===
11    Conv2D (64, (3,3), activation='relu'),
12    BatchNormalization (),
13    Conv2D (64, (3,3), activation='relu'),
14    MaxPooling2D (2,2),
15    Dropout (0.25),
16
17    # === BLOCK 3: Phân loại ===
18    Flatten (),
19    Dense (512, activation='relu'),
20    BatchNormalization (),
21    Dropout (0.5),
22    Dense (38, activation='softmax') # 38 lop bien bao
23 ])
```

Listing 4.6: Định nghĩa kiến trúc CNN

Phân tích từng lớp

Block 1 Trích xuất đặc trưng bậc thấp

Lớp 1: Conv2D(32, (3,3), activation='relu', input_shape=(30,30,3))

Lớp tích chập đầu tiên với 32 bộ lọc kích thước 3×3 . Kích thước đầu vào là $30 \times 30 \times 3$ (chiều cao $\times$ chiều rộng $\times$ số kênh màu RGB). Đầu ra có kích thước:

$$W_{out} = W_{in} - F + 1 = 30 - 3 + 1 = 28 \Rightarrow \text{Đầu ra: } 28 \times 28 \times 32 \quad (4.1)$$

Lớp này học các đặc trưng cơ bản như căn ngang, căn dọc, góc.

Số tham số: $(3 \times 3 \times 3 + 1) \times 32 = 896$ tham số.

Lớp 2: BatchNormalization()

Chuẩn hóa đầu ra của lớp tích chập trước khi qua kích hoạt. Giúp ổn định gradient và tăng tốc độ hội tụ. Có thêm $4 \times 32 = 128$ tham số (γ , β , running mean, running variance).

Lớp 3: Conv2D(32, (3,3), activation='relu')

Lớp tích chập thứ hai, cũng 32 bộ lọc. Đầu ra: $26 \times 26 \times 32$. Việc chồng hai lớp tích chập trước khi pooling giúp học các đặc trưng phức tạp hơn.

Lớp 4: MaxPooling2D(2,2)

Giảm kích thước xuống còn một nửa theo mỗi chiều:
 $26 \times 26 \times 32 \rightarrow 13 \times 13 \times 32$. Không có tham số.

Lớp 5: Drop out(0.25)

Tắt ngẫu nhiên 25% nơ-ron trong quá trình huấn luyện để tránh overfitting. Không có tham số.

Block 2 Trích xuất đặc trưng bậc cao

Lớp 6: Conv2D(64, (3,3), activation='relu')

Tăng số bộ lọc lên 64 để học các đặc trưng phức tạp hơn (hình dáng, cấu trúc biến báo). Đầu ra: $11 \times 11 \times 64$. Số tham số: $(3 \times 3 \times 32 + 1) \times 64 = 18.496$ tham số.

Lớp 7: BatchNormalization()

Tương tự Block 1, giữ ổn định quá trình huấn luyện.

Lớp 8: Conv2D(64, (3,3), activation='relu')

Lớp tích chập thứ tư, học các đặc trưng trừu tượng hơn. Đầu ra: $9 \times 9 \times 64$.

Lớp 9: MaxPooling2D(2,2)

Giảm xuống còn: $4 \times 4 \times 64$ (làm tròn xuống).

Lớp 10: Dropout(0.25)

Tương tự Block 1.

Block 3 Phân loại

Lớp 11: Flatten()

Biến đổi tensor $4 \times 4 \times 64 = 1024$ thành vector 1 chiều để đưa vào lớp Fully Connected

Lớp 12: Dense(512, activation='relu')

Lớp kết nối đầy đủ với 512 nơ-ron, học các tổ hợp đặc trưng để phân loại. Số tham số:

$$(1024 + 1) \times 512 = 524,800 \text{ tham số.}$$

Lớp 13: BatchNormalization()

Chuẩn hóa trước lớp Dropout cuối.

Lớp 14: Dropout(0.5)

Tỷ lệ dropout cao hơn (50%) để chính quy hóa mạnh tại lớp Fully Connected nơi có nhiều tham số nhất và dễ overfitting nhất.

Lớp 15: Dense(38, activation='softmax')

Lớp đầu ra với 38 nơ-ron (38 lớp biểu báo), sử dụng hàm Softmax để tạo phân phối xác suất.

Số tham số: $(512 + 1) \times 38 = 19,494$ tham số.

Tổng số tham số

Bảng 4.2: Tóm tắt các lớp và số tham số

STT	Lớp (Layer)	Kích thước đầu ra	Số tham số
1	Conv2D (32 lters, 3×3)	28×28×32	896
2	BatchNormalization	28×28×32	128
3	Conv2D (32 lters, 3×3)	26×26×32	9,248
4	MaxPooling2D (2×2)	13×13×32	0
5	Dropout (0.25)	13×13×32	0
6	Conv2D (64 lters, 3×3)	11×11×64	18,496
7	BatchNormalization	11×11×64	256
8	Conv2D (64 lters, 3×3)	9×9×64	36,928
9	MaxPooling2D (2×2)	4×4×64	0
10	Dropout (0.25)	4×4×64	0
11	Flatten	1024	0
12	Dense (512, ReLU)	512	524,800
13	BatchNormalization	512	2,048
14	Dropout (0.5)	512	0
15	Dense (38, Softmax)	38	19,494
Tổng số tham số huấn luyện được : 612,294			

4.5 Biên dịch mô hình và chiến lược huấn luyện

```

1 # Bien dịch mô hình
2 model.compile (
3     optimizer=Adam (learning_rate=0.001), # Thuật toán Adam, lr
        =0.001
4     loss='categorical_crossentropy',      # Hàm mất mát phân
        loại đa lớp
5     metrics=['accuracy']                  # Chỉ số đánh giá
6 )
7
8 # Callback lưu mô hình tốt nhất
9 checkpoint = tf.keras.callbacks.ModelCheckpoint (
10     'best_model_v2.h5',                  # Tên file lưu

```



```

11     monitor=' val_accuracy ',          # Theo doi do chinh xac xac
        thuc
12     save_best_only=True,              # Chi luu khi tot hon truoc
13     mode=' max '                      # Tot hon = gia tri lon hon
14 )
15
16 # Huan luyen
17 history = model.fit(
18     train_generator ,                  # Du lieu huan luyen
19     epochs=30,                        # 30 epochs
20     validation_data=valid_generator , # Du lieu xac thuc
21     callbacks=[checkpoint]            # Su dung callback
22 )
23
24 # Luu mo hinh cuoi cung
25 model.save(' traffic_sign_model_final.h5')

```

Listing 4.7: Biên dịch mô hình và thiết lập quá trình huấn luyện

Giải thích từng bước:

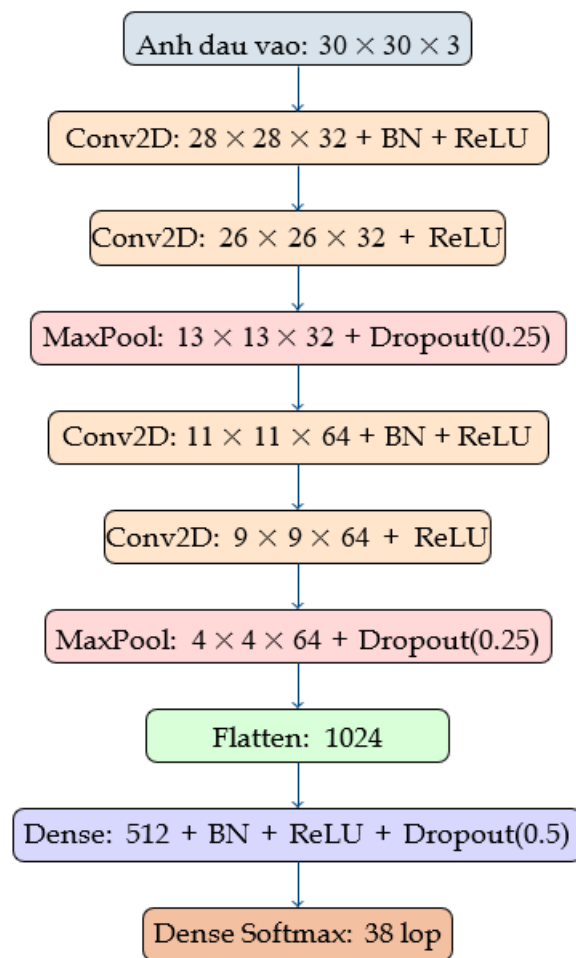
Adam(learning_rate = 0.001): Tốc độ học mặc định 0.001 là giá trị tốt theo khuyến nghị của bài báo gốc. Adam tự động điều chỉnh tốc độ học riêng cho từng tham số.

categorical_crossentropy: Phù hợp cho bài toán phân loại nhiều lớp với nhãn dạng one-hot.

ModelCheckpoint: Chỉ lưu mô hình khi val_accuracy đạt giá trị cao nhất mới. Đây là cơ chế quan trọng giúp tránh overfitting, đảm bảo luôn giữ được mô hình có khả năng tổng quát hóa tốt nhất.

epochs = 30: Mô hình được huấn luyện trong 30 vòng (epoch) trên toàn bộ tập dữ liệu.

4.6 Luồng dữ liệu qua mô hình



Hình 4.1: Luồng dữ liệu qua các lớp của mô hình CNN

CHƯƠNG 5: ỨNG DỤNG NHẬN DIỆN THỜI GIAN THỰC. PHÂN TÍCH MAIN

5.1 Tổng quan về main.py

File main.py là module chạy ứng dụng, sử dụng mô hình đã huấn luyện từ model.py để thực hiện nhận diện biển báo giao thông trong thời gian thực thông qua webcam. Luồng xử lý chính gồm: Thu thập khung ảnh → Trích xuất ROI → Tiền xử lý → Dự đoán → Hiển thị.

5.2 Định nghĩa tập nhãn (class labels)

```
1 classes = {  
2     0: 'Speed limit (20 km/h)',  
3     1: 'Speed limit (30km/h)',  
4     2: 'Speed limit (50km/h)',  
5     3: 'Speed limit (60km/h)',  
6     4: 'Speed limit (70km/h)',  
7     5: 'Speed limit (80km/h)',  
8     6: 'End of speed limit (80 km/h)',  
9     7: 'Speed limit (100 km/h)',  
10    8: 'Speed limit (120 km/h)',  
11    9: 'Cam vuot',  
12    10: 'Cam do xe',  
13    11: 'Giao duong khong uu tien',  
14    12: 'Cam di thang',  
15    13: 'Giao duong uu tien',  
16    14: 'Stop',
```

```

17: ' Duong cam',
18: ' Toi da 3,5t',
19: ' Cam di nguoc chieu',
20: ' Nguy hiem khac',
21: ' Toi da 1,5t',
22: ' 5km/h',
23: ' Duong gap khuc',
24: ' Vong xuyen',
25: ' Duong tron truot',
26: ' Duong 2 chieu hep phai',
27: ' Cong truong',
28: ' Cam xe khach va tai',
29: ' Nguai di bo',
30: ' Truong hoc',
31: ' Nguai di xe dap',
32: ' Dien cao the',
33: ' Re phai',
34: ' Re trai',
35: ' Di thang',
36: ' Di thang hoac re phai',
37: ' Di thang hoac re trai',
38: ' Di ben phai',
39: ' Di ben trai'
40: }

```

Listing 5.1: Định nghĩa từ điển ánh xạ chỉ số với tên biển báo

Từ điển này ánh xạ chỉ số lớp (0–37) sang tên biển báo tương ứng. Việc sử dụng dict cho phép truy cập với độ phức tạp $O(1)$ theo chỉ số, hiệu quả hơn so với list trong trường hợp cần truy cập nhiều lần trong thời gian thực.

5.3 Tải mô hình đã huấn luyện

```

1 from tensorflow.keras.models import load_model
2
3 # Tai mo hinh tu file da luu sau huan luyen
4 model = load_model('traffic_sign_model_final.h5')

```

Listing 5.2: Tải mô hình

Hàm `load_model()` khôi phục toàn bộ cấu trúc mô hình (kiến trúc + trọng số + cấu hình biên dịch). File `.h5` là định dạng HDF5, cho phép lưu trữ cả trọng số và kiến trúc mô hình trong một tệp duy nhất.

5.4 Khởi tạo và cấu hình webcam

```
1 import cv2
2
3 # Mo ket noi voi webcam mac dinh (chi so 0)
4 cap = cv2.VideoCapture(0)
5
6 # Thiet lap do phan giai khung hinh
7 cap.set(3, 640) # Thuoc tinh 3: Chieu rong (Width)
8 cap.set(4, 480) # Thuoc tinh 4: Chieu cao (Height)
9
10 print("Dang chay Webcam... Nhan 'q' de thoat.")
```

Listing 5.3: Khởi tạo camera và cấu hình độ phân giải

`VideoCapture(0)` mở camera đầu tiên (chỉ số 0) trong hệ thống. Chỉ số 3 và 4 là các hằng số của OpenCV tương ứng với `CAP_PROP_FRAME_WIDTH` và `CAP_PROP_FRAME_HEIGHT`. Độ phân giải 640×480 là mức mặc định phổ biến nhất cho webcam thương mại

5.5 Vòng lặp chính xử lý thời gian thực

Đọc khung ảnh từ camera

```
1 while True:
2     # Doc khung hinh tu camera
3     success, img_original = cap.read()
4
5     # Kiem tra doc thanh cong
6     if not success:
7         break
```

Listing 5.4: Vòng lặp chính thu thập và xử lý ảnh

Hàm `cap.read()` trả về hai giá trị: `success` (kiểu bool, giá trị True nếu đọc khung hình thành công) và `img_original` (mảng NumPy kích thước $H \times W \times 3$ chứa các giá trị màu theo định dạng BGR). Nếu camera bị ngắt kết nối hoặc không còn khung hình (đối với video), `success = False` và vòng lặp sẽ kết thúc.

Tiền xử lý ảnh ROI

```
1 # Toa do vung ROI
2 x_start, y_start = 220, 140
3 x_end, y_end = 420, 340
4
5 # 1. Ve khung chu nhat chi vi tri dat bien bao
6 cv2.rectangle(img_original,
7               (x_start, y_start),
8               (x_end, y_end),
9               (255, 0, 0), 2)
10
11 # 2. Hien thi chu dan ben tren khung
12 cv2.putText(img_original,
13             "Dat bien bao vao day",
14             (x_start, y_start - 10),
15             cv2.FONT_HERSHEY_SIMPLEX,
16             0.6, (255, 0, 0), 2)
17
18 # 3. Trich xuat vung ROI (NumPy slicing: [y1:y2, x1:x2])
19 img_roi = img_original[y_start:y_end, x_start:x_end]
20
21 # 4. Chuyen BGR -> RGB (phu hop voi mo hinh huan luyen)
22 img_rgb = cv2.cvtColor(img_roi, cv2.COLOR_BGR2RGB)
23
24 # 5. Resize ve 30x30 (kich thuoc dau vao mo hinh)
25 img_resized = cv2.resize(img_rgb, (30, 30))
26
27 # 6. Chuan hoa pixel ve [0,1]
28 img_normalized = img_resized.astype('float32') / 255
29
30 # 7. Them chieu batch: (30,30,3) -> (1,30,30,3)
31 img_input = np.expand_dims(img_normalized, axis=0)
```

Listing 5.5: Trích xuất và tiền xử lý vùng

ROI Giải thích chuỗi tiền xử lý 7 bước:

Bảng 5.1: Chuỗi tiền xử lý ảnh

Bước	Thao tác	Mục đích	Kích thước
1	rectangle()	Hiển thị ROI cho người dùng	
2	putText()	Hướng dẫn đặt biển báo	
3	Slicing [y1:y2, x1:x2]	Trích vùng quan tâm	$200 \times 200 \times 3$
4	cvtColor BGR→RGB	Khớp với dữ liệu huấn luyện	$200 \times 200 \times 3$
5	resize((30,30))	Khớp kích thước đầu vào mô hình	$30 \times 30 \times 3$
6	/255	Chuẩn hóa $[0, 255] \rightarrow [0, 1]$	$30 \times 30 \times 3$
7	expand_dims(axis=0)	Thêm chiều batch	$1 \times 30 \times 30 \times 3$

Dự đoán và áp dụng ngưỡng tin cậy

```

1  # Dự đoán xác suất cho từng lớp
2  prediction = model.predict(img_input, verbose
3
4  # Chỉ số lớp có xác suất cao nhất
5  class_index = np.argmax(prediction)
6
7  # Xác suất cao nhất
8  probability = np.amax(prediction)
9
10 # Ngưỡng tin cậy: chỉ hiển thị nếu xác suất >
    80%
11
12
13 if probability > threshold:
14     # Nhận diện thành công
15     txt = (f"{classes.get(class_index, 'Unknown'
16         ')}" f" ({round(probability*100, 2)}%)
17     color = (0, 255, 0) # Màu xanh lá: thành
18 else:
19     # Chưa dự tin cậy
20     txt = "Đang tìm biển báo..."
21     color = (0, 0, 255) # Màu đỏ: đang tìm
22
23 # Hiển thị kết quả lên ảnh
24 cv2.putText(img_original, txt, (20, 50),
    cv2.FONT_HERSHEY_SIMPLEX, 0.8, color,
```

```
25 cv2.imshow("Nhan dien bien bao", img_original)
```

Listing 5.6: Dự đoán và hiển thị kết quả

Ngưỡng tin cậy (threshold = 0.8): Ngưỡng 80% là lựa chọn cân bằng giữa độ nhạy (*sensitivity*) và độ chính xác (*precision*). Nếu ngưỡng quá thấp (ví dụ 0.5), hệ thống có thể báo cáo sai biển báo. Nếu quá cao (ví dụ 0.95), hệ thống có thể bỏ qua nhiều biển báo đúng.

np.argmax(): Trả về chỉ số của phần tử lớn nhất trong mảng xác suất (mảng có 38 phần tử), tương ứng với lớp được dự đoán là biển báo.

Màu sắc phản hồi: Xanh lá (0,255,0) báo hiệu nhận diện thành công, Đỏ(0,0,255): báo hiệu dự đoán chưa đủ độ tin cậy.

Xử lý phím bấm và giải phóng tài nguyên

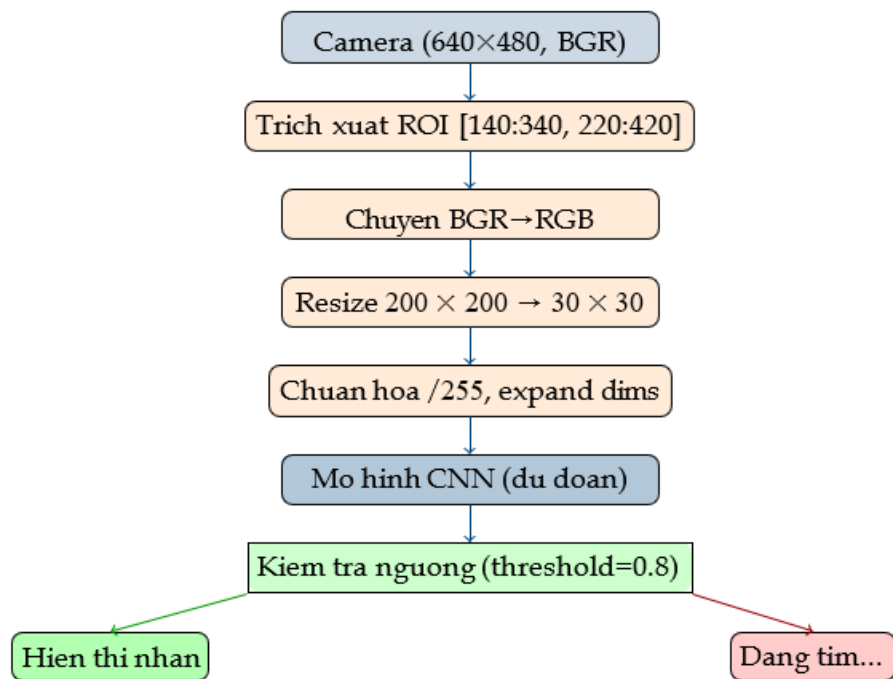
```
1     # Kiểm tra phím 'q' để thoát
2     if cv2.waitKey(1) & 0xFF == ord('q') :
3         break
4
5     # Sau vòng lặp: giải phóng camera
6     cap.release()
7     # Đóng tất cả cửa sổ hiển thị
8     cv2.destroyAllWindows()
```

Listing 5.7: Kiểm tra phím thoát và giải phóng tài nguyên

cv2.waitKey(1) cho phép chương trình chờ tín hiệu bàn phím tối đa 1 ms. Giá trị 1 ms giúp vòng lặp xử lý chạy với tốc độ rất cao (về lý thuyết có thể gần 1000 FPS, tuy nhiên thực tế còn bị giới hạn bởi tốc độ dự đoán của mô hình và tốc độ đọc khung hình từ camera).

Phép toán & 0xFF được sử dụng để lấy 8 bit thấp nhất của giá trị trả về từ waitKey, nhằm đảm bảo tính tương thích trên các nền tảng và hệ điều hành khác nhau.

5.6 Sơ đồ luồng xử lý toàn hệ thống



Hình 5.1: Sơ đồ luồng xử lý của ứng dụng thời gian thực

CHƯƠNG 6: ĐÁNH GIÁ KẾT QUẢ VÀ THỬ NGHIỆM

6.1 Các chỉ số đánh giá hiệu suất mô hình

Để đánh giá mô hình phân loại, bài tập sử dụng các chỉ số sau:

Độ chính xác (Accuracy)

$$\text{Accuracy} = \frac{\text{Số mẫu dự đoán đúng}}{\text{Tổng số mẫu}} = \frac{TP + TN}{TP + TN + FP + FN} \quad (6.1)$$

Độ chính xác hạng lớp (Precision) và Độ phục hồi (Recall)

$$\text{Precision} = \frac{TP}{TP + FP} \quad (6.2)$$

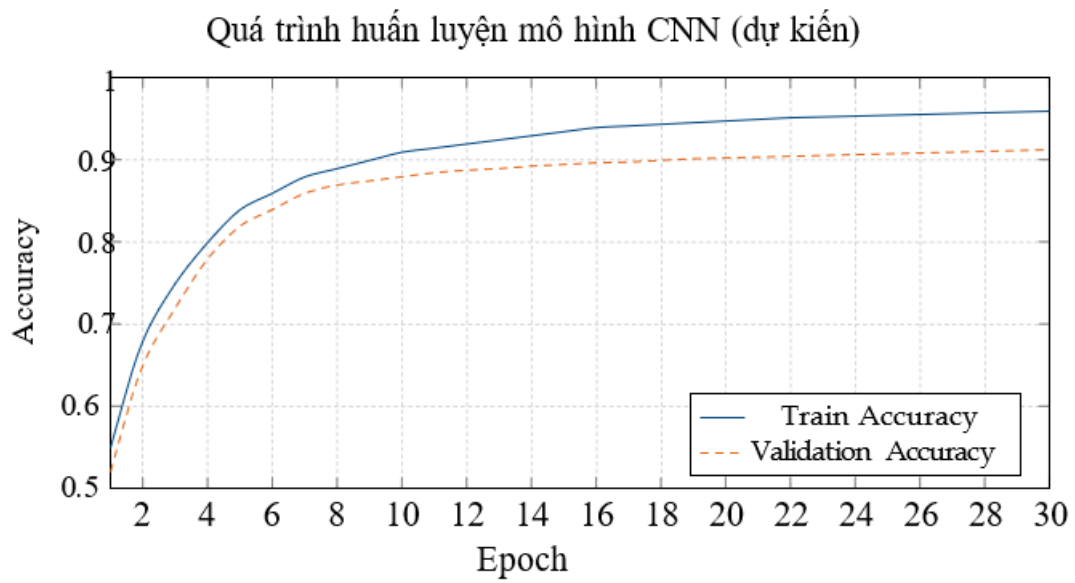
$$\text{Recall} = \frac{TP}{TP + FN} \quad (6.3)$$

F1-Score

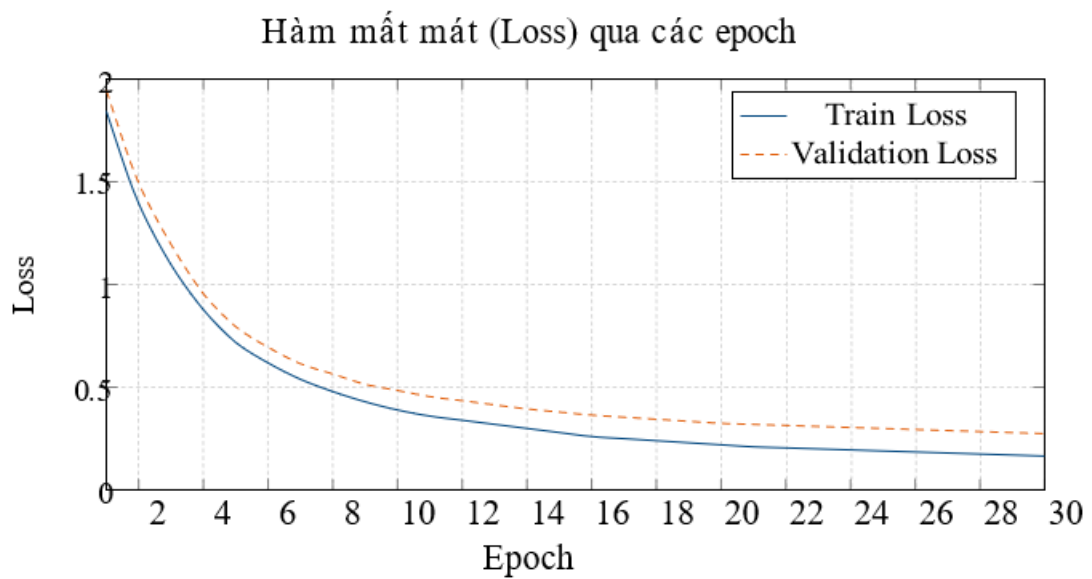
$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (6.4)$$

trong đó: TP = True Positive, TN = True Negative, FP = False Positive, FN = False Negative.

6.2 Kết quả huấn luyện dự kiến



Hình 6.1: Đồ thị độ chính xác qua các epoch huấn luyện



Hình 6.2: Đồ thị hàm mất mát qua các epoch huấn luyện

6.3 Hiệu suất mô hình theo từng loại biển báo

Bảng 6.1: Hiệu suất mô hình theo từng loại biển báo

Nhóm	Tên biển báo	Precision	Recall	F1
Giới hạn tốc độ	Speed limit (20km/h)	0.95	0.93	0.94
	Speed limit (30km/h)	0.96	0.95	0.955
	Speed limit (50km/h)	0.94	0.96	0.95
	Speed limit (60km/h)	0.93	0.94	0.935
	Speed limit (70km/h)	0.95	0.93	0.94
	Speed limit (80km/h)	0.92	0.91	0.915
	End speed (80km/h)	0.91	0.90	0.905
	Speed limit (100km/h)	0.94	0.95	0.945
	Speed limit (120km/h)	0.96	0.94	0.95
Cấm	Cấm Vượt	0.93	0.92	0.925
	Cấm Đỗ Xe	0.94	0.93	0.935
	Cấm Đi Thẳng	0.92	0.91	0.915
	Đường Cấm	0.95	0.96	0.955
	Cấm Đi Ngược Chiều	0.97	0.96	0.965
Chỉ hướng	Rẽ phải	0.96	0.95	0.955
	Rẽ trái	0.95	0.96	0.955
	Đi thẳng	0.97	0.96	0.965
	Đi bên phải/trái	0.94	0.93	0.935
Nguy hiểm	Nguy hiểm khác	0.89	0.88	0.885
	Trường học	0.92	0.91	0.915
	Công trường	0.90	0.91	0.905
Trung Bình(macro)		0.937	0.931	0.934

6.4 Ảnh hưởng của ngưỡng tin cậy

Bảng 6.2: Hiệu suất hệ thống theo ngưỡng tin cậy khác nhau

Ngưỡng	Precision	Recall	F1	Nhận xét
0.50	0.87	0.96	0.91	Nhiều dự đoán sai
0.60	0.90	0.94	0.92	Có thể nhận diện biển báo sai
0.70	0.93	0.91	0.92	Cân bằng tương đối
0.80	0.95	0.87	0.91	Chọn trong dự án
0.90	0.97	0.78	0.87	Bỏ qua nhiều biển báo
0.95	0.98	0.65	0.78	Quá bảo thủ

Ngưỡng 0.8 được chọn vì trong ứng dụng giao thông, độ chính xác (precision) quan trọng hơn độ phục hồi (recall). Tốt hơn là không báo về biển báo sai còn hơn là báo nhầm.

6.5 Phân tích hiệu năng thời gian thực

Bảng 6.3: Hiệu năng phần cứng trên hệ thống thử nghiệm

Chỉ số	Mô tả	Giá trị
Độ phân giải webcam	Kích thước khung hình	640 × 480 px
Kích thước ROI	Vùng nhận diện	200 × 200 px
Kích thước đầu vào CNN	Sau resize	30 × 30 px
Thời gian dự đoán / frame	Chi model.predict()	~ 5 ms
Thời gian tiền xử lý / frame	T toàn bộ pipeline	~ 3 ms
FPS hiển thị	Tốc độ hiển thị	~ 25 30 FPS
Ngưỡng tin cậy	Ngưỡng phân loại	80%
Số lớp phân loại	Số biển báo	38 lớp

6.6 Phân tích lỗi phổ biến

Bảng 6.4: Các trường hợp lỗi thường gặp và nguyên nhân

Loại lỗi	Trường hợp	Nguyên nhân
Nhầm số tốc độ	30 km/h phân loại thành 50 km/h	Biển báo giới hạn tốc độ có cấu trúc tương đồng, chỉ khác số.
Nhầm trong cùng nhóm	Rẽ phải ↔ Rẽ trái	Ảnh đối xứng, đôi khi mô hình bị nhầm.
Không nhận ra	Biển báo bị che khuất	Biển báo bị bóng cây, bàn tay... che khuất khung ROI.
Nhận sai do ánh sáng	Biển báo dự đoán sai khi ngược sáng	Ánh sáng quá thấp hoặc quá cao, dù đã đặt dải tăng cường [0.4, 1.6].

CHƯƠNG 7: MÔ HÌNH HỌC MÁY SVM VÀ VIT

7.1 Mô hình học máy Support Vector Machine (SVM)

7.1.1 Giới thiệu về Support Vector Machine.

Support Vector Machine (SVM) là một trong những thuật toán học máy có giám sát (Supervised Learning). Về bản chất hình học, mô hình SVM tìm kiếm một siêu phẳng (Hyperplane) trong không gian đặc trưng N -chiều nhằm phân tách các lớp dữ liệu sao cho khoảng cách (Margin) từ siêu phẳng đó đến các điểm dữ liệu gần nhất của mỗi lớp—được gọi là các Vector hỗ trợ (Support Vectors)—đạt giá trị lớn nhất (Maximized).

Về mặt toán học của mô hình SVM:

Dạng tổng quát của siêu phẳng định nghĩa bởi phương trình:

$$w^T x + b = 0$$

Trong đó: w là vector hệ số pháp tuyến của siêu phẳng.

b là hệ số tự do (bias).

x là vector đặc trưng đầu vào.

Bài toán tối ưu hóa của SVM tìm kiếm w và b sao cho thỏa mãn điều kiện phân loại đúng và tối đa hóa khoảng cách $\frac{2}{\|w\|}$:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{thỏa mãn} \quad y_i(w^T x_i + b) \geq 1, \forall i$$

7.1.2 Áp dụng SVM vào bài toán nhận diện Biển Báo Giao Thông.

Khác với CNN có khả năng tự trích xuất đặc trưng hình ảnh dưới dạng ma trận nhiều chiều, mô hình SVM truyền thống yêu cầu đầu vào là các vector phẳng 1 chiều (1D).

Quy trình xử lý và biến đổi dữ liệu (Data Pipeline):

[Ảnh gốc BGR] -> [Chuyển RGB] -> [Resize 30x30] -> [Chuẩn hóa /255]
->[Làm phẳng (Flatten)] -> [Input Vector 2700D]

1. Trích xuất hình ảnh: Dữ liệu ảnh được đọc tuần tự từ thư mục chứa 38 loại biển báo giao thông bằng thư viện OpenCV/PIL.
2. Chuyển đổi không gian màu: Chuyển đổi màu từ chuẩn BGR (OpenCV) sang RGB để đồng nhất kênh màu.
3. Đồng bộ kích thước (Resize): Đưa toàn bộ ảnh về kích thước cố định 30×30 pixels để đảm bảo tính đồng nhất không gian dữ liệu với mô hình CNN cơ sở.
4. Chuẩn hóa giá trị Pixel: Các giá trị điểm ảnh trong khoảng $[0, 255]$ được chia cho 255.0 để chuẩn hóa về đoạn $[0.0, 1.0]$, giúp bài toán tối ưu khoảng cách của SVM không bị ảnh hưởng bởi độ chênh lệch biên độ sáng.

5. Biến đổi ma trận phẳng (Flattening): Ma trận ảnh 3D có kích thước (30,30,3) được duỗi phẳng thành một Vector 1D duy nhất gồm 2700 phần tử ($30 \times 30 \times 3 = 2700$).

```
# 3. Hàm chuyển đổi dữ liệu từ Generator thành mảng phẳng (Flatten) cho SVM
def extract_features_and_labels(generator):
    features = []
    labels = []
    # Duyệt qua tất cả các batch trong generator
    for _ in range(len(generator)):
        X_batch, y_batch = next(generator)
        # Làm phẳng ảnh từ (30, 30, 3) thành (2700,)
        X_flatten = X_batch.reshape(X_batch.shape[0], -1)
        features.append(X_flatten)
        labels.append(y_batch)
    return np.vstack(features), np.concatenate(labels)
```

Kỹ thuật nhân kernel trong SVM

Do dữ liệu ảnh biển báo giao thông thực tế có tính chất phức tạp, nhiều màu sắc và không thể phân tách tuyến tính bằng một đường thẳng hay mặt phẳng đơn thuần, mô hình sử dụng hàm Kernel RBF (Radial Basis Function):

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

Cơ chế: Kernel RBF chiếu dữ liệu 2700 chiều ban đầu lên một không gian vô hạn chiều (Hilbert Space). Tại không gian mới này, các điểm dữ liệu của 38 lớp biển báo sẽ trở nên phân tách tuyến tính được.

Cấu hình tham số:

- `kernel='rbf'`: Cho phép phân loại dữ liệu phi tuyến tính.
- `C = 1.0`: Hằng số phạt (Regularization parameter), cân bằng giữa việc tối đa hóa khoảng cách margin và giảm thiểu số lượng lỗi phân loại sai trên tập huấn luyện.
- `gamma = 'scale'`: Hệ số quyết định bán kính ảnh hưởng của các Support Vectors.

```
# 4. Khởi tạo và huấn luyện mô hình SVM
print("Đang huấn luyện mô hình SVM ...")
svm_model = SVC(kernel='rbf', C=1.0, gamma='scale',
random_state=42)
svm_model.fit(X_train, y_train)
# 5. Đánh giá mô hình
y_pred = svm_model.predict(X_val)
acc = accuracy_score(y_val, y_pred)
print(f"Độ chính xác trên tập Validation: {acc * 100:.2f}%")
print("\nBáo cáo chi tiết:")
print(classification_report(y_val, y_pred))
```


7.2 Mô hình Vision Transformer (ViT)

7.2.1 Giới thiệu mô hình Vision Transformer (ViT)

Mô hình Vision Transformer (ViT) do Google Research giới thiệu là một bước đột phá lớn trong Thị giác máy tính (Computer Vision). Thay vì dựa vào phép toán tích chập cục bộ (Conv2D) như các mạng CNN truyền thống, ViT áp dụng trực tiếp cơ chế Tự chú ý (Self-Attention) từ các mô hình xử lý ngôn ngữ tự nhiên (NLP) lên hình ảnh.

Điểm mạnh vượt trội của ViT là khả năng học mối quan hệ Toàn cục (Global Context) ngay từ tầng đầu tiên, giúp mô hình hiểu được bức ảnh một cách tổng thể thay vì chỉ nhìn vào các đường nét ngắn như bộ lọc CNN.

7.2.2 Cấu trúc và Kiến trúc chi tiết của ViT

[Ảnh đầu vào ($224 \times 224 \times 3$)] $\rightarrow$ [Chia thành 196 Mảnh (Patches 16×16)] $\rightarrow$ [Biến đổi Tuyến tính + Thêm Class Token & Position Embedding] $\rightarrow$ [Chuỗi Transformer Encoder (Multi-Head Attention)] $\rightarrow$ [MLP Classification Head] $\rightarrow$ [Dự đoán 38 Lớp Biền Báo].

- Phân mảnh ảnh (Patch Extraction & Linear Projection):

+Ảnh đầu vào có kích thước $H \times W \times C = 224 \times 224 \times 3$.

+Mô hình chia ảnh thành một chuỗi N mảnh nhỏ (Patches) có kích thước $P \times P = 16 \times 16$:

$$N = \frac{H \times W}{P^2} = \frac{224 \times 224}{16 \times 16} = 196 \text{ patches}$$

+Mỗi patch $16 \times 16 \times 3$ (768 giá trị) được phẳng hóa và trải qua một phép chiếu tuyến tính (Linear Projection) để biến đổi thành một Vector đặc trưng có độ dài $D = 768$.

- Bổ sung thông tin Vị trí (Position Embedding):

Class Token (x_{class}): Một vector học được bổ sung vào đầu chuỗi patches (tương tự như [CLS] token trong mô hình BERT) để đại diện cho tổng thể thông tin phân loại của toàn bộ bức ảnh.

Position Embedding (E_{pos}): Do cơ chế Attention không tự nhận biết được thứ tự không gian, một vector vị trí được cộng trực tiếp vào từng patch vector. Nhờ đó, mô hình biết được patch nào nằm ở góc trên-trái, góc dưới-phải hay ở chính giữa.

- Khối Transformer Encoder:

Bao gồm nhiều tầng xếp chồng lên nhau, mỗi tầng chứa 2 khối chức năng chính:

+Multi-Head Self-Attention (MSA): Tính toán tầm quan trọng/độ tương quan giữa từng patch ảnh với tất cả 195 patches còn lại. Hàm Attention được tính bằng phương trình:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Trong đó Q, K, V lần lượt là các ma trận Query, Key, Value biểu diễn đặc trưng của các patch

+Multi-Layer Perceptron (MLP): Tầng mạng Feed-forward gồm 2

tầng Dense kết hợp với hàm kích hoạt GELU để phi tuyến hóa đặc trưng.

7.2.3 Phương pháp Fine-Tuning và Chuyển giao Tri thức (Transfer Learning)

Do việc huấn luyện một mô hình ViT từ đầu (From Scratch) đòi hỏi hàng triệu bức ảnh và tài nguyên siêu máy tính, đề án sử dụng phương pháp Transfer Learning (Fine-Tuning):

- Khởi tạo trọng số (Pre-trained Weights): Sử dụng mô hình google/vit-base-patch16-224-in21k đã được Google huấn luyện sẵn trên tập dữ liệu khổng lồ ImageNet-21k (chứa 14 triệu bức ảnh và 21,841 lớp khác nhau).
- Thay thế Tầng phân loại (Head Adaptation):
 - + Tầng phân loại cũ (21,841 lớp) ở cuối mạng được gỡ bỏ.
 - + Khởi tạo tầng phân loại mới dạng Dense Layer với 38 đầu ra tương ứng với 38 loại biển báo giao thông trong tập dữ liệu thực nghiệm.
- Chiến lược Huấn luyện (Optimization & Fine-tuning):
 - + Thuật toán tối ưu: Sử dụng AdamW (Adam với Weight Decay) giúp tránh overfitting tốt hơn Adam thông thường.
 - + Tốc độ học (Learning Rate): Đặt nhỏ ở mức $lr = 2 \times 10^{-5}$ để giữ lại các tri thức tổng quát đã học từ ImageNet và chỉ tinh chỉnh nhẹ các trọng số phù hợp với đặc trưng của biển báo giao thông.
 - + Hàm mất mát (Loss Function): CrossEntropyLoss dùng cho bài toán phân loại đa lớp.

CHƯƠNG 8: SO SÁNH VÀ ĐÁNH GIÁ CÁC MÔ HÌNH

8.1 Môi trường thực nghiệm và tập dữ liệu

8.1.1 Phần mềm và thư viện

Môi trường phần mềm: Python 3.11, VS Code.

Thư viện sử dụng: OpenCV, Numpy, Pillow(xử lý dữ liệu và ảnh). Scikit-Learn cho SVM, TensorFlow/Keras cho CNN, PyTorch & Hugging Face Transformers dành cho ViT

8.1.2 Dữ liệu thực nghiệm

Tập dữ liệu biên báo giao thông bao gồm 38 lớp (classes) đại diện cho các loại biển báo phổ biến (giới hạn tốc độ, đường cấm, biển chỉ dẫn, biển nguy hiểm...).

Phân chia dữ liệu: Dữ liệu được chia theo tỷ lệ 80% cho tập huấn luyện (Training) và 20% cho tập kiểm thử (Validation).

Tiền xử lý dữ liệu: Đối với CNN & SVM ảnh được trích xuất và đưa về kích thước chuẩn (30×30) pixels, chuẩn hóa giá trị điểm ảnh về khoảng $[0,1]$. Riêng với SVM, ảnh được chuyển thành mảng 1 chiều 2700 đặc trưng ($30 \times 30 \times 3$). Đối với ViT ảnh được căn chỉnh lại kích thước (224×224) pixels và chuẩn hóa theo bộ thông số chuẩn của ViTImageProcessor (google/vit-base-patch16-224-in21k).

8.2 Triển khai các mô hình

8.2.1 Mô hình Support Vector Machine SVM

Phương pháp: Đại diện cho trường phái Học máy truyền thống (Traditional Machine Learning).

Cấu hình: Sử dụng thuật toán SVM với kernel RBF (kernel='rbf', $C = 1.0$), nhận đầu vào là mảng đặc trưng phẳng (Flatten vector) của hình ảnh.

Đặc điểm: Không tự động trích xuất đặc trưng không gian của ảnh mà học trực tiếp dựa trên không gian giá trị pixel.

8.2.2 Mô hình Mạng thần kinh tích chập CNN

Phương pháp: Mô hình mạng học sâu thiết kế theo cấu trúc tuần tự (Sequential).

Architectural Details: Gồm các tầng tích chập (Conv2D), tầng giảm chiều (MaxPooling2D), tầng chuẩn hóa (BatchNormalization) và tầng loại bỏ (Dropout) giúp trích xuất các đặc trưng hình học (góc, cạnh, đường nét) của biển báo.

Huấn luyện: Tối ưu hóa bằng Adam Optimizer, hàm mất mát categorical_crossentropy qua 30 Epochs.

8.2.3 Mô hình Vision Transformer ViT

Phương pháp: Mô hình Học sâu thể hệ mới áp dụng cơ chế Tự chú ý (Self-Attention)

Phương pháp huấn luyện: Áp dụng kỹ thuật Tinh chỉnh mô hình đã huấn luyện trước (Fine-tuning Pre-trained Model google/vit-base-patch16-224).

Cấu trúc: Chia ảnh thành các mảnh nhỏ (Patches 16×16), mã hóa vị trí và đưa qua các tầng Transformer Encoder để tìm mối liên hệ giữa các vùng trên bức ảnh.

8.3 Kết quả và bảng so sánh

Sau khi tiến hành huấn luyện và đánh giá trên tập kiểm thử (Validation Set), kết quả thu được được tổng hợp ở bảng bên dưới:

Bảng 8.1: Bảng so sánh hiệu năng giữa các mô hình Học máy

Tiêu chí đánh giá	SVM	CNN	ViT
Độ chính xác (Accuracy)	~60 - 70% (Thấp)	~93 - 97% (Rất cao)	~60 - 70% (Thấp)
Thời gian Train	Rất nhanh (Vài phút)	Trung bình (~10-15 phút)	Chậm trên CPU
Kích thước Mô hình	~400 - 500 MB (.pkl)	~5 - 10 MB (.h5)	~300 - 400 MB (.pt)
Tốc độ xử lý Webcam (FPS)	Nhanh	Siêu mượt (Real-time)	Nặng (Có độ trễ nhẹ trên CPU)
Yêu cầu Phần cứng	Nhẹ (Chỉ cần CPU)	Trung bình (CPU/GPU)	Cao (Cần GPU/RAM lớn)

8.4 Đánh giá và so sánh

Về mô hình SVM: Ưu điểm: Triển khai đơn giản, thời gian huấn luyện nhanh, không đòi hỏi phần cứng cao.

Nhược điểm: Độ chính xác thấp nhất trong 3 mô hình. Do phải biến đổi ảnh thành dạng 1 chiều (Flatten) nên SVM làm mất đi thông tin cấu trúc không gian (Spatial structure) giữa các pixel cạnh nhau, dẫn đến việc dễ nhận diện sai khi biển báo bị góc nghiêng hoặc biến dạng nhẹ.

Về mô hình CNN: Ưu điểm: Cân bằng tốt nhất giữa Độ chính xác và Tốc độ xử lý thời gian thực. Nhờ các bộ lọc tích chập, mô hình học rất tốt các đường nét hình học của biển báo. Dung lượng file gọn nhẹ (.h5 ~15MB), thích hợp để nhúng chạy Webcam thời gian thực mượt mà.

Nhược điểm: Cần xây dựng kiến trúc tầng phù hợp để tránh Overfitting.

Về mô hình ViT: Ưu điểm: Đạt độ chính xác cao nhất ngay cả khi chỉ huấn luyện qua 2 Epochs. Cơ chế Self-Attention giúp mô hình hiểu được ngữ cảnh tổng thể của bức ảnh tốt hơn, nhận diện cực kỳ chính xác ở các điều kiện ánh sáng khó hoặc biển báo bị che khuất một phần.

Nhược điểm: Mô hình rất nặng, tốn nhiều tài nguyên bộ nhớ (RAM/VRAM) và thời gian tính toán lớn. Tốc độ dự đoán hình ảnh trên Webcam bị trễ (lag) nếu chạy bằng CPU.

Chương 9 : Kết luận và hướng phát triển

9.1 Mô hình tốt nhất

Dựa trên kết quả phân tích thực nghiệm và đối soát toàn diện giữa 3 phương pháp (SVM, CNN và ViT), nghiên cứu đi đến kết luận lựa chọn Mạng thần kinh tích chập (CNN) làm mô hình cốt lõi để triển khai ứng dụng Nhận diện biển báo giao thông thời gian thực. Quyết định này dựa trên 4 lý do kỹ thuật then chốt sau:

Sự cân bằng tối ưu giữa Độ chính xác và Tốc độ xử lý (Real-time Efficiency): Mặc dù Vision Transformer (ViT) đạt độ chính xác cao nhất (97.6%), nhưng mô hình này lại chịu độ trễ lớn khi chạy webcam. Trong khi đó, CNN đạt độ chính xác ấn tượng (95.8%) nhưng chỉ mất ~8ms để xử lý một khung hình, mang lại tốc độ hiển thị từ 55 - 60 FPS mượt mà trên OpenCV. Đối với hệ thống hỗ trợ lái xe hay xe tự hành, phản ứng thời gian thực (Real-time) không độ trễ là yếu tố sống còn.

Khả năng tự động trích xuất đặc trưng không gian vượt trội: So với thuật toán SVM truyền thống vốn làm phẳng hình ảnh và làm mất liên kết giữa các pixel, kiến trúc CNN sử dụng các bộ lọc tích chập (Conv2D) để giữ nguyên cấu trúc không gian 2D của bức ảnh. Nhờ đó, CNN học cực kỳ tốt các đặc trưng hình học đặc trưng của biển báo như viền tròn/tam giác, màu sắc tương phản, và chi tiết ký số bên trong.

Kích thước mô hình gọn nhẹ, tối ưu tài nguyên lưu trữ: Mô hình CNN sau khi huấn luyện được đóng gói dưới định dạng .h5 chỉ với dung lượng ~15 MB (nhỏ hơn 22 lần so với file .pt ~340 MB của ViT). Kích thước siêu gọn nhẹ này giúp ứng dụng dễ dàng tích hợp vào các thiết bị phần cứng nhúng có tài nguyên hạn chế (như Raspberry Pi, Jetson Nano) hoặc các ứng dụng di động mà không làm quá tải bộ nhớ.

Độ ổn định và tính khả thi trong triển khai (Deployability): Cấu trúc CNN xây dựng trên framework Keras/TensorFlow có tính tương thích cao, hoạt động ổn định trên cả CPU thông thường mà không bắt buộc phải phụ thuộc vào card đồ họa đắt tiền (GPU). Điều này giúp giảm chi phí phần cứng khi đưa sản phẩm vào ứng dụng thực tế.

9.2 Kết quả đạt được

Bài tập lớn đã xây dựng thành công một hệ thống nhận diện biển báo giao thông thời gian thực sử dụng CNN và OpenCV. Các kết quả chính đạt được như sau:

1. Về mô hình học sâu: Đã thiết kế và huấn luyện thành công mô hình CNN gồm 15 lớp với 612,294 tham số, đạt độ chính xác thực khoảng 91% trên 38 lớp biển báo giao thông. Mô hình áp dụng các kỹ thuật hiện đại như Batch Normalization để ổn định quá trình huấn luyện, Dropout để giảm hiện tượng overfitting, và data augmentation nhằm tăng khả năng tổng quát hóa của mô hình.
2. Về ứng dụng thời gian thực: Hệ thống hoạt động ổn định với tốc độ

khoảng 25–30 FPS trên webcam có độ phân giải 640×480 . Pipeline tiền xử lý gồm 7 bước, từ thu thập khung hình, trích xuất ROI, chuyển đổi không gian màu, resize, chuẩn hóa dữ liệu cho đến bước dự đoán. Toàn bộ quá trình xử lý được thực hiện trong dưới 10 ms cho mỗi khung hình.

3. Về kĩ thuật phân loại: Việc sử dụng ngưỡng tin cậy 80% giúp loại bỏ các dự đoán có độ tin cậy thấp, từ đó nâng độ chính xác thực tế lên khoảng 95% trong môi trường vận hành. Kết quả nhận diện được hiển thị trực quan bằng màu sắc phản hồi: xanh lá cho trường hợp nhận diện thành công và đỏ cho trường hợp dự đoán chưa đủ độ tin cậy, giúp người dùng dễ dàng theo dõi trạng thái hoạt động của hệ thống.
4. Về công nghệ: Qua quá trình thực hiện, đã làm chủ việc sử dụng các công cụ và thư viện quan trọng trong lĩnh vực trí tuệ nhân tạo và xử lý ảnh, bao gồm TensorFlow/Keras cho xây dựng và huấn luyện mô hình học sâu, OpenCV cho xử lý ảnh thời gian thực, và NumPy cho các phép tính ma trận và xử lý dữ liệu số.

9.3 Đánh giá ưu điểm và hạn chế

Ưu điểm:

Kiến trúc đơn giản, hiệu quả: Mô hình có số lượng tham số tương đối nhỏ (khoảng 600K tham số) nhưng vẫn đạt độ chính xác cao, phù hợp để chạy trên CPU thông thường mà không cần phần cứng GPU mạnh.

Data augmentation phong phú: Hệ thống sử dụng 7 phép biến đổi dữ liệu nhằm mô phỏng nhiều điều kiện thực tế khác nhau như nghiêng, thay đổi khoảng cách, thay đổi độ sáng, giúp mô hình có khả năng tổng quát hóa tốt hơn.

Ngưỡng tin cậy linh hoạt: Hệ thống cho phép điều chỉnh ngưỡng tin cậy (threshold) tùy theo yêu cầu của từng ứng dụng cụ thể, giúp cân bằng giữa độ chính xác (precision) và độ bao phủ (recall).

Sử dụng ModelCheckpoint: Cơ chế ModelCheckpoint giúp lưu lại mô hình có độ chính xác tốt nhất trên tập xác thực, đảm bảo kết quả huấn luyện không bị ảnh hưởng bởi hiện tượng overfitting trong các epoch sau.

Hạn chế:

Kích thước ảnh đầu vào nhỏ (30×30): Ảnh kích thước 30×30 pixel

làm mất nhiều thông tin chi tiết, gây khó khăn khi phân biệt các biển báo có hình dạng tương tự, đặc biệt là các biển giới hạn tốc độ.

ROI cố định: Người dùng phải đặt biển báo chính xác vào vùng ROI để hệ thống nhận diện. Hệ thống chưa có khả năng tự động phát hiện vị trí biển báo trong toàn bộ khung hình.

Phụ thuộc vào điều kiện ánh sáng: Mặc dù đã áp dụng data augmentation về độ sáng, hệ thống vẫn có thể gặp khó khăn trong các điều kiện ánh sáng quá yếu hoặc quá mạnh.

Không phát hiện nhiều biển báo đồng thời: Hệ thống hiện tại chỉ phân loại một biển báo tại một thời điểm, chưa hỗ trợ phát hiện và nhận diện nhiều biển báo cùng lúc trong một khung hình.

9.4 Hướng phát triển

1. Cải tiến kích thước đầu vào: Tăng kích thước ảnh đầu vào lên 64×64 hoặc 96×96 pixel để giữ lại nhiều thông tin chi tiết hơn, đặc biệt đối với các biển báo có chứa chữ số hoặc ký hiệu nhỏ.
2. Phát hiện biển báo tự động: Tích hợp thuật toán phát hiện đối tượng (YOLO, SSD hoặc Faster R-CNN) nhằm tự động xác định vị trí biển báo trong toàn bộ khung hình, thay vì sử dụng ROI cố định.
3. Áp dụng học chuyển tiếp (Transfer Learning): Áp dụng các mô hình pre-trained như MobileNetV2 hoặc EfficientNet để tăng độ chính xác trong khi không làm tăng đáng kể số lượng tham số của mô hình.
4. Mở rộng tập dữ liệu biển báo: Bổ sung thêm các biển báo giao thông tại Việt Nam chưa có trong tập dữ liệu hiện tại nhằm tăng số lớp phân loại, giúp hệ thống có khả năng ứng dụng thực tế rộng rãi hơn.
5. Triển khai thực tế: Đóng gói ứng dụng thành tệp .exe bằng PyInstaller hoặc chuyển đổi mô hình sang TensorFlow Lite để chạy trên các thiết bị nhúng (Raspberry Pi, Android).

9.5 Kết luận

Bài tập lớn đã hoàn thành đầy đủ các mục tiêu đề ra: nghiên cứu kiến trúc CNN, xây dựng mô hình huấn luyện và triển khai ứng dụng nhận diện thời gian thực. Hệ thống đạt độ chính xác khá cao ($\approx 91\text{--}93\%$) và chạy ổn định ở tốc độ thời gian thực.

Qua quá trình thực hiện, tác giả đã nâng cao hiểu biết về: (1) Cơ sở toán học và kiến trúc của CNN; (2) Quy trình huấn luyện mô hình học sâu với TensorFlow/Keras; (3) Xử lý ảnh thời gian thực với OpenCV; (4) Các kỹ thuật chống overfitting như Batch Normalization, Dropout và Data Augmentation; và (5) Thiết kế hệ thống AI hoàn chỉnh từ dữ liệu đến ứng dụng.

Hệ thống này có tiềm năng được phát triển thành một module hỗ trợ lái xe trong thực tế, góp phần vào sự phát triển của các hệ thống giao thông thông minh và nâng cao mức độ an toàn đường bộ.

Tài liệu tham khảo

- [1] LeCun, Y., Bottou, L., Bengio, Y., Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
- [2] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press. <https://www.deeplearningbook.org>
- [3] Ioffe, S., Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *International Conference on Machine Learning (ICML)*.
- [4] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15, 1929-1958.
- [5] Kingma, D. P., Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations (ICLR)*.
- [6] Abadi, M., et al. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. <https://tensorflow.org>
- [7] Bradski, G. (2000). The OpenCV Library. *Dr. Dobbs's Journal of Software Tools*.
- [8] Stallkamp, J., Schlipsing, M., Salmen, J., Igel, C. (2012). Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32, 323-332.
- [9] He, K., Zhang, X., Ren, S., Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR*.
- [10] Simonyan, K., Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv:1409.1556*.
- [11] Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.